Certified reliability for any routing solver: screening buffered plans against travel-time uncertainty, with evidence from real delivery routes

Suhan Ali
Independent researcher, Lahore, Pakistan
syedsuhanali9@gmail.com ORCID 0009-0004-5079-7684

Abstract

Delivery firms promise time windows but plan routes with fixed travel times, and padding every trip, the usual protection, says nothing about how reliable the plan is. We add a reliability layer to an existing routing solver without changing it. The solver plans with internal deadlines brought forward by up to an hour, capped so that no customer becomes unreachable; the plans are simulated under correlated, time-dependent delays; and the chosen plan is certified on independent simulated days with an exact binomial bound. We show that a certified plan misses the reliability target with probability at most 5%, whatever the solver and selection rule. Across a development study, three confirmatory studies with published hypotheses, benchmark instances and 40 Amazon delivery routes with delays fitted to New York taxi trips, none of 666 certified plans failed its certificate on fresh simulated days, while 20% padding was never certified on a synthetic instance. A plan reaching 95% reliability existed on 46 of 53 instances with buffers and on 23 with padding from 0 to 50%, using fewer vehicles. The certificate is only as good as the delay model: it held when real delays were milder than assumed and often failed when they were harsher.

Keywords: vehicle routing; time windows; travel-time uncertainty; simulation; chance constraints; reliability certification

1 Introduction

A courier company that promises to deliver between 10:00 and 11:00 must decide which driver visits which customers, and in what order, so that every visit falls inside its window. This is the vehicle routing problem with time windows (VRPTW; Solomon, 1987; Toth and Vigo, 2014). Routing software usually solves it with one fixed table of travel times, but real travel times change with the time of day and with random events, and because a driver serves customers one after another, an early delay pushes every later visit towards the end of its window.

The common protection is to plan with a slower speed than expected, for example assuming that every trip takes 20% longer. Padding is simple and works with any solver, but it spends the same protection on quiet and congested trips, and it says nothing about the result: a planner who pads by 20% does not know whether the plan will be on time on 60% of days or on 99%. Methods that model the uncertainty directly, through chance constraints, recourse or robust optimization, spend protection more carefully (Laporte et al., 1992; Li et al., 2010; Errico et al., 2018; Agra et al., 2013; Ehmke et al., 2015), but they need a solver built for that model, which most firms do not have and cannot easily adopt. A third family,

simheuristics, embeds Monte Carlo simulation in a metaheuristic and ranks solutions by their simulated cost and reliability (Juan et al., 2015; Abdullahi et al., 2025); it too is tied to its own search algorithm, and the simulated reliability it reports is an estimate from the same scenarios that guided the search.

This paper takes the systems view of a firm that already has a routing solver and wants a reliability statement it can act on. We wrap the existing solver in a layer with three steps. The solver is asked for a small family of plans whose internal deadlines, which customers never see, are brought forward by a buffer between 0 and 60 minutes. Each plan is simulated on 1000 days with time-of-day congestion and correlated random delays, and one plan is selected. The selected plan is then simulated on 5000 further days that were not used to produce or choose it, and it is *certified* if an exact binomial lower bound on its probability of a successful day reaches the target.

The paper makes four contributions.

1. *A solver-independent certificate.* We show, by an elementary argument borrowed from sampled chance-constrained programming, that the probability that a certified plan misses the reliability target is at most the chosen error level, for any solver, candidate family, selection rule or fallback, including a simheuristic used as the solver (Theorem 1). On fresh simulated days, none of the 666 certified plans in our studies failed its certificate, and we measure how the certificate fares when the delay model is wrong.

2. *A reachability cap.* Bringing deadlines forward can make a distant customer impossible to reach; we give a capped deadline that never does this (Lemma 1), and show that it removes most screening failures and also repairs a budgeted robust optimization model.

3. *The cost of reliability.* For every instance we compare the cheapest plan that reaches a given reliability in the buffer family with the cheapest one reachable by padding from 0 to 50%. Buffers reach 95% on twice as many instances and use fewer vehicles where both reach it.

4. *Evidence beyond synthetic data.* Besides a development study and three confirmatory studies whose hypotheses were published before they were run, the layer is tested on standard benchmark instances and on 40 real Amazon delivery routes, with the delay model fitted to New York taxi trips. We also test what happens when the delay model used for screening is wrong.

Two limits frame all of this. The certificate is a statement about the simulated delay model; Section 6.7 measures how far it carries over when the model is wrong. And the delivery windows on the Amazon routes are assumed, because the data set contains few.

2 Related work

Routing with random travel times has been studied for more than three decades (Laporte et al., 1992; Gendreau et al., 1996; Kenyon and Morton, 2003); Gendreau et al. (2016) survey the open problems. One line of work states on-time service as a chance constraint and optimizes it directly. Li et al. (2010) combine chance constraints and recourse with a tabu search, Ehmke et al. (2015) embed per-customer service levels in a routing heuristic with approximated arrival-time distributions, Taş et al. (2014) combine time-dependent and stochastic travel times with soft windows, and Bomboi et al. (2022) develop feasibility checks for correlated, time-dependent travel times. Errico et al. (2018) solve the problem with stochastic

service times exactly. Robust and risk-based models replace the probability with an uncertainty set or a risk index: Agra et al. (2013) solve a robust VRPTW with budgeted uncertainty sets in the sense of Bertsimas and Sim (2004), and Jaillet et al. (2016) and Zhang et al. (2019) route to minimize indices of how far arrival times can breach their windows. Closest in structure to the present paper, Zhang et al. (2021) minimize a service fulfilment risk index under a Wasserstein ambiguity set built from travel-time samples, and show that the problem reduces to the sample version with every deadline brought forward by a buffer. Their probability bound holds over the ambiguity set, the buffer is set by its radius, and out-of-sample performance is assessed empirically, within a dedicated exact or heuristic solver. Serrano et al. (2026) learn travel-time distributions from context. All of these build the uncertainty model into a specialist optimizer.

The internal deadlines used here are closest to self-imposed time windows, in which the carrier sets its own windows inside the promised ones so that slack absorbs delays (Jabali et al., 2015). Vareias et al. (2019) assess customer service reliability for such windows with stochastic travel times, solving routing and window placement in two levels. Hosseini et al. (2025) design the windows offered to customers for a given route under correlated travel times. In our setting the promised windows are fixed, only an internal planning deadline moves, and the buffer is one number per plan rather than a window per customer.

Simheuristics (Juan et al., 2015) are the closest methodological relative. They take a metaheuristic for the deterministic problem, simulate promising solutions, and use the simulated cost or reliability to steer and rank the search; Abdullahi et al. (2025) add an explicit reliability analysis for a sustainable routing problem with stochastic travel times and demands. Our layer differs in two ways. It does not change the search: any solver, including a simheuristic, can produce the candidates. And it separates estimation from decision. The days used to make and choose the plan are never used to certify it, which is what makes an exact, finite-sample guarantee possible.

That separation is standard in stochastic programming. In sample approximations of chance-constrained programs, a candidate solution found on one sample is checked for feasibility on an independent sample (Luedtke and Ahmed, 2008; Ahmed and Shapiro, 2008), and the scenario approach gives a priori guarantees for convex problems (Campi and Garatti, 2008). We apply the a posteriori version to routing plans, where the problem is not convex and the guarantee must not depend on the solver. The exact binomial bound of Clopper and Pearson (1934) gives the certificate; the Wilson bound (Wilson, 1927), which behaves well near one (Brown et al., 2001), is used for the cheaper screen.

3 Problem and model

3.1 Routing model

Let $N = \{0, 1, \ldots, n\}$ be the nodes, with depot 0 and customers $C = N \setminus \{0\}$, and let V be a homogeneous fleet with capacity Q and maximum shift length H. Customer i has demand δ_i, service time s_i and a promised window $[l_i, u_i]$ in which service must start. The nominal distance and planning time of arc (i, j) are d_{ij} and $\bar{t}_{ij}$. With binary arc variables x_{ijv}, assignment variables z_{iv} and vehicle-use variables y_v, service

start times a_{iv}, depot departure and return times d_v, r_v, and slack deficits $G_{iv} \geq 0$, a plan satisfies

$$\sum_{v \in V} z_{iv} = 1, \quad \sum_j x_{ijv} = \sum_j x_{jiv} = z_{iv} \ (i \in C), \quad \sum_{j \in C} x_{0jv} = \sum_{i \in C} x_{i0v} = y_v, \quad \sum_{i \in C} \delta_i z_{iv} \leq Q y_v, \tag{1}$$

$$l_i z_{iv} \leq a_{iv} \leq u_i z_{iv}, \qquad a_{jv} \geq a_{iv} + s_i + \bar{t}_{ij} - M(1 - x_{ijv}), \qquad a_{jv} \geq d_v + \bar{t}_{0j} - M(1 - x_{0jv}), \tag{2}$$

$$r_v \geq a_{iv} + s_i + \bar{t}_{i0} - M(1 - x_{i0v}), \qquad 0 \leq d_v \leq r_v \leq H y_v, \qquad G_{iv} \geq a_{iv} - (u_i - s^*) z_{iv}, \tag{3}$$

for all $v \in V$, where M is a sufficiently large constant and $s^* = 15$ minutes is a target slack before the end of each window. The time constraints exclude subtours. Four criteria describe a plan: distance cost $F_1 = c \sum d_{ij} x_{ijv}$, slack deficit $F_2 = \sum G_{iv}$, driver time $F_3 = \sum_v (r_v - d_v)$ and vehicles used $F_4 = \sum_v y_v$. The planner minimizes

$$Z = \sum_{k=1}^{4} w_k \frac{F_k - L_k}{\max\{U_k - L_k, 1\}}, \tag{4}$$

where L_k and U_k are the smallest and largest values of F_k among four solutions that each minimize one criterion, and $w = (0.5693, 0.2643, 0.1055, 0.0609)$ is the principal eigenvector of an Analytic Hierarchy Process comparison matrix (Saaty, 1980) with consistency ratio 0.025, fixed before any experiment. The weights only shape and rank candidates; the screen and the certificate do not use them. The solver never builds (2) explicitly: OR-Tools (Perron and Furnon, 2026) enforces the timing through its routing dimensions, and the hybrid genetic search (HGS) of PyVRP (Vidal, 2022; Wouda et al., 2024) is used as a second, stronger solver. Travel times are rounded up to whole minutes.

3.2 Travel times and delays

Each simulated day starts at 06:00 with a free-flow speed of 40 km/h, which is divided by 0.9 before 07:00, 1.2 from 07:00 to 09:00, 1.0 from 09:00 to 17:00, 1.3 from 17:00 to 19:00 and 0.9 afterwards. Travel times integrate the speed over the trip, so a later departure never arrives earlier (Ichoua et al., 2003; Ghiani and Guerriero, 2014). On each day, the time of arc (i, j) is multiplied by $\exp(\varepsilon_{ij})$ with

$$\varepsilon_{ij} = \sigma\big(\sqrt{\rho}\,\xi_0 + \sqrt{1-\rho}\,\xi_{ij}\big), \tag{5}$$

where ξ_0 is a standard normal variable common to all trips of the day and the ξ_{ij} are independent standard normal variables. The assumed setting is $\sigma = 0.2$ and $\rho = 0.5$; Section 5.4 fits these quantities and the time-of-day profile to real traffic. Days are independent and identically distributed. A fixed set of simulated days is a *bank*; on each instance all plans are judged on the same banks, so comparisons are paired.

3.3 Reliability

A day is a *success* for a plan if at least $\kappa = 95\%$ of its customers start service by the end of their promised windows. Requiring every customer to be on time would be unrealistic on large instances: with 200 customers each on time with probability 0.99 independently, all would be on time on only 13% of days. The *reliability* of plan x is the probability $p(x)$ of a successful day, and the target is $p(x) \geq \tau = 0.95$. A plan's reliability is a property of the plan and the delay model, not of any particular bank.

4 Screen and certify

4.1 Buffered candidates and the reachability cap

For each buffer $\beta \in \{0, 5, \ldots, 60\}$ minutes the solver plans against internal deadlines $u_i - \beta$, never earlier than l_i, which gives up to 13 candidate plans. Simulated arrivals are always judged against the promised u_i. A large buffer can, however, put a distant customer's internal deadline before the earliest time it can be reached, and then no plan exists. Let $e_i = \max\{l_i, \bar{t}_{0i}\}$ be the earliest possible start of service at i after a direct trip from the depot. The capped deadline is

$$u_i^{\mathrm{cap}}(\beta) = \min\bigl\{u_i,\ \max\{l_i,\ u_i - \beta,\ e_i\}\bigr\}. \tag{6}$$

Lemma 1. *For every customer i and buffer $\beta \geq 0$: (a) $u_i^{\mathrm{cap}}(\beta) \leq u_i$, so a plan that meets the capped deadlines meets the promised windows; (b) if $e_i \leq u_i$, then $e_i \leq u_i^{\mathrm{cap}}(\beta)$, so the cap never makes a reachable customer unreachable; (c) if $\beta \leq u_i - e_i$, then $u_i^{\mathrm{cap}}(\beta) = \max\{l_i, u_i - \beta\}$, so the cap changes nothing where it is not needed.*

Proof. (a) The outer minimum includes u_i. (b) The inner maximum is at least e_i and $u_i \geq e_i$, so their minimum is at least e_i. (c) $\beta \leq u_i - e_i$ gives $u_i - \beta \geq e_i$, so the inner maximum is $\max\{l_i, u_i - \beta\} \leq u_i$, and the outer minimum leaves it unchanged. □

The lemma protects each customer separately. It does not guarantee that one plan can serve all customers in time, since capacity, shift length and visit order still matter. A shortest-path bound, with service times included, was used to prove where possible that a contracted model had no feasible plan, so that solver failures could be told apart from true infeasibility.

4.2 Screening

Every candidate is simulated on a screening bank of 1000 days. If it succeeds on a share $\hat{p}$ of them, its one-sided 95% Wilson lower bound is

$$L_W(\hat{p}) = \frac{\hat{p} + \frac{z^2}{2N} - z\sqrt{\frac{\hat{p}(1-\hat{p})}{N} + \frac{z^2}{4N^2}}}{1 + \frac{z^2}{N}}, \qquad z = 1.645,\ N = 1000. \tag{7}$$

A candidate passes if $L_W \geq \tau$. With $\hat{p} = 0.966$, for example, $L_W \approx 0.955$ and the candidate passes; with $\hat{p} = 0.955$, $L_W \approx 0.943$ and it fails. Among passing candidates the one with the smallest score (4) is selected. If none passes, a fallback fixed in advance is returned. The screen is a filter, not a guarantee: with 13 candidates tested on the same days, the chance that some weak candidate passes is larger than 5%.

4.3 The certificate

The selected plan is simulated once on a validation bank of $N_c = 5000$ days drawn independently of everything used to produce and select it. Let K be its number of successful days and $L_{CP}(K)$ the exact one-sided $(1-\alpha)$ Clopper–Pearson lower bound, the α-quantile of the Beta$(K, N_c - K + 1)$ distribution (Clopper and Pearson, 1934). The plan is *certified* if $L_{CP}(K) \geq \tau$; with $\alpha = 0.05$ this requires at least 4776 successful days out of 5000.

Theorem 1. *Let a plan X be produced by any procedure, deterministic or random, that may use the instance, any solver and any simulated days other than the validation bank. Let the validation days be independent of that procedure and identically distributed under the delay model. Then*

$$\Pr\big(X \text{ is certified and } p(X) < \tau\big) \le \alpha.$$

Proof. Condition on the information $\mathscr{F}$ used by the procedure, which fixes X. Because the validation days are independent of $\mathscr{F}$ and identically distributed, the successes on them are independent Bernoulli trials with probability $p(X)$, so $K \sim \text{Binomial}(N_c, p(X))$ given $\mathscr{F}$. If $p(X) \ge \tau$, the event in question is empty. If $p(X) < \tau$, certification requires $L_{CP}(K) \ge \tau > p(X)$, and the Clopper–Pearson bound satisfies $\Pr\{L_{CP}(K) > p\} \le \alpha$ for every true probability p. Hence the conditional probability of the event is at most α, and taking the expectation over $\mathscr{F}$ gives the claim. □

Remark 1. The argument is elementary: it is the a posteriori feasibility check used for sampled chance-constrained programs (Luedtke and Ahmed, 2008), applied to a routing plan. Its value here is practical. It places no condition on the solver, the buffer family, the selection rule or the fallback; the candidates may even be chosen by a simheuristic that has simulated thousands of scenarios of its own. It does not say that a plan is reliable on real roads: $p(X)$ is defined by the delay model. Section 6.7 tests that gap.

Corollary 1. *If the K_c candidates are produced without using the screening bank, admitting a candidate only when its Clopper–Pearson bound at level α/K_c on the screening bank reaches τ gives the same guarantee for the admitted plan without a separate validation bank, by the union bound over candidates.*

In this paper the screen uses the Wilson bound and the guarantee comes from the separate certificate. An earlier exploratory check of the Bonferroni form of the screen (Corollary 1 with the Wilson bound in place of the exact one) removed all 11 admitted plans that later fell short, at a cost of at most 2 admissions per study.

4.4 Configurations compared

Table 1 defines every configuration. The *capped procedure* is the main version of the layer. *Screened conservative* combines the screen with padding. The padded plans are used as an operator would use them, without screening.

5 Experimental design

5.1 Synthetic instances and confirmatory studies

Synthetic instances place a depot at the centre of a 100 km square and customers uniformly at random, with demands from 10 to 49 units, windows 90 to 149 minutes wide, capacity $Q = 1000$, service time 10 minutes and a 600-minute shift. The development study used 53 instances: 20 with 20 customers, 15 with 50, 10 with 100 and 8 with 200. Each OR-Tools call had 5 seconds and up to three attempts; each HGS call had 10,000 iterations. The development study was exploratory and shaped the method. Three confirmatory studies then ran on new instances, each with its hypotheses, tests, decision rules and seeds written down and publicly timestamped with a fingerprint of the code before any instance was solved (Ali,

Table 1. Configurations. Every plan is judged against the promised windows on the same simulated days.

Name	Definition
Nominal plan	Shortest distance at 40 km/h, no buffer, no screening.
Padding $p\%$	Shortest distance at $40/(1+p/100)$ km/h, no screening; *20% padding*, also called conservative speed, is $p = 20$.
Slack-aware plan	Weighted objective (4), which includes the slack deficit, with no buffer and no screening.
Original procedure	Buffered candidates without the cap, weighted objective, screened; fallback: the slack-aware plan.
Capped procedure	As the original procedure with the capped deadlines (6); the HGS version falls back to 20% padding.
Screened conservative	Buffered candidates planned at the 20% padded speed for distance, screened, shortest passing candidate selected; fallback: 20% padding.
Best-screened fallback	When no candidate passes, return the candidate with the highest Wilson bound (tested in study 3B).
Robust model	Budgeted robust VRPTW (Agra et al., 2013; Bertsimas and Sim, 2004), with an analogous reachability correction, screened in the same way.

2026b,c,d). Study 1 tested the OR-Tools procedures on 53 new instances, study 2 the HGS procedure on 53 more, and study 3 compared the procedures with the robust model on the 90 earlier confirmatory instances with at most 100 customers (part A) and tested the best-screened fallback on 53 further instances (part B). Each study or part had two primary hypotheses, judged with 97.5% paired bootstrap intervals so that the chance of a false confirmation stays below 5% per pair. All other analyses are exploratory.

5.2 Benchmark instances

The HGS procedure was also run on the 20 Gehring–Homberger instances of classes R1 and RC1 with 200 customers (Gehring and Homberger, 1999), which extend Solomon's test set (Solomon, 1987). Coordinates and times were rescaled so that the benchmark's unit-speed travel matches 40 km/h and its horizon fits a 600-minute shift from 06:00, and the same delay model was added.

5.3 Real delivery routes

The third set comes from the public data of the 2021 Amazon Last Mile Routing Research Challenge (Merchán et al., 2022). We drew 40 routes that Amazon rated as high quality, with a fixed seed and at most five per depot, from 17 depots in five US cities. Each route is one van's day, with 80 to 194 stops (median 150). The data give the travel time between every pair of stops, the planned service time and volume of every package, the van's capacity and departure time, and the order in which the driver actually drove. Consecutive stops in the same Amazon delivery zone were joined into one customer, giving 10 to 36 customers per route, because rounding every trip up to a whole minute would add more than an hour to a route of 150 stops that are often less than a minute apart. A customer's service time, its packages plus the travel between its stops, was added to the trips leaving it, so service receives the same random delays as driving. Only 7% of these stops have a delivery window in the data, and the middle half of those windows are 7 to 8 hours wide, so each customer was offered a two-hour window containing the time the driver actually arrived, placed at random around it. The shift starts at the real local departure time and lasts 1.3 times the actual route, and up to three vans of the real capacity may be used. The unchanged OR-Tools pipeline was run on every route.

Table 2. Paired differences in service probability (percentage points) on the 46 development instances where 20% padding found a plan. C: 20% padding; S: the slack-aware plan (weighted objective, no buffer, no screening); P: the original procedure; SC: screened conservative (Table 1).

n	S minus C	P minus C	SC minus C
20	−11.0 [−16.1, −6.0]	21.4 [9.6, 31.3]	28.2 [21.4, 34.3]
50	−13.1 [−17.4, −8.9]	17.9 [2.8, 32.0]	11.8 [2.7, 22.2]
100	−16.2 [−20.6, −11.7]	16.0 [4.8, 24.8]	11.2 [2.8, 19.4]
200	−12.9 [−18.5, −5.7]	8.3 [−5.1, 20.0]	5.7 [0.0, 12.9]

5.4 Delays fitted to real traffic

To replace the assumed delay model with one estimated from data, we used the yellow taxi trip records published by New York City (New York City Taxi and Limousine Commission, 2024). Of the 9.6 million trips from January to March 2024, 6.2 million weekday trips between two different taxi zones passed basic checks (0.5 to 30 miles, 1 to 120 minutes, 1 to 60 miles per hour). We used each trip's time per mile so that trips of different length between the same zones are comparable. The time-of-day multipliers compare the same pairs of zones in each period with 09:00–17:00. The spread σ was estimated in two ways: from all variation between trips on the same pair of zones and period, which includes differences between addresses inside a zone and so overstates the variation of one fixed trip, and from the variation between days only, which understates it. The day-common share ρ is the variance of the daily mean divided by each. The Amazon experiment was repeated under both fitted models, with the screen and the certificate simulating the fitted model.

5.5 Analyses

Pooled service is the mean validated reliability over all instances, an instance without a plan scoring zero, so planning failures count against a configuration. Differences are paired by instance, with bootstrap intervals from 20,000 resamples. Three further analyses use only stored plans. To test the certificate, every certified plan was simulated on a third bank of 5000 fresh days; a plan *fails its certificate* if the exact one-sided 95% upper bound on its fresh reliability is below τ, which the theorem allows for about 5% of certified plans at most. To measure the cost of reliability, every buffer candidate of the development study was judged on the validation bank, alongside the padded plans at 0 to 50% (Section 6.4). To test a wrong delay model, plans chosen under one model were judged under another (Section 6.7).

6 Results

6.1 Screening against 20% padding

Table 2 compares the procedures with 20% padding on the 46 development instances where padding found a plan. The original procedure was 8.3–21.4 percentage points more reliable, depending on size; at 200 customers the interval includes zero. The slack-aware plan without screening was worse than padding at every size, so the gain comes from testing candidates, not from the objective. The advantage grew with the amount of random delay (Figure 1): with independent delays it rose from -3.1 points at $\sigma = 0.1$, where padding was slightly ahead, to 30.5 points at $\sigma = 0.3$.

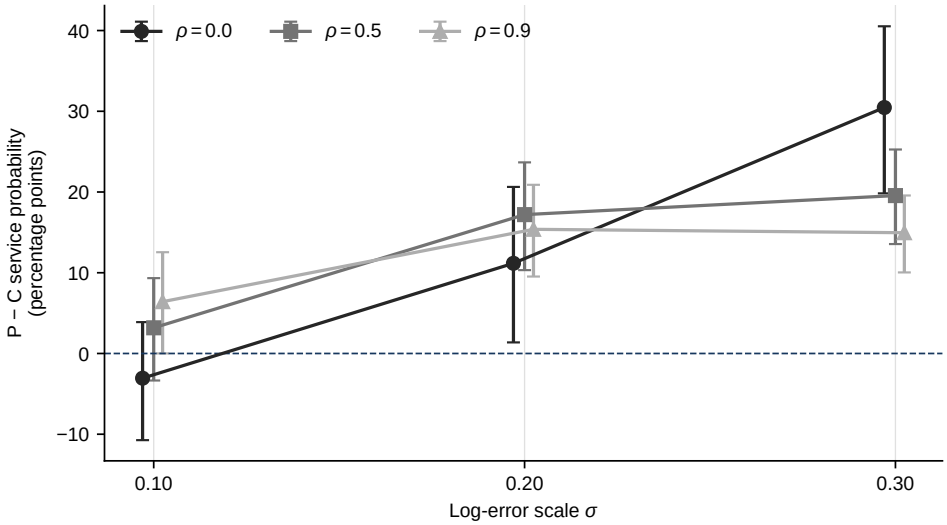


Figure 1. Difference in validated reliability between the original procedure and 20% padding on the 46 development instances, as the delay spread σ and the day-common share ρ vary. Plans are fixed; error bars are 95% bootstrap intervals over instances.

Table 3. Development study, all 53 instances. For each procedure: instances admitted by screening out of all instances of that size, with the number whose lower bound also reached 0.95 on the validation bank in parentheses, and pooled validated service probability (%) with a 95% bootstrap interval, fallback plans included.

	Original procedure		Screened conservative		Capped procedure	
n	Admitted	Pooled	Admitted	Pooled	Admitted	Pooled
20	14/20 (14)	81.6 [70.2, 91.9]	15/20 (15)	87.7 [79.0, 94.7]	14/20 (14)	81.6 [70.3, 92.0]
50	8/15 (8)	72.7 [59.2, 86.4]	4/15 (4)	67.5 [58.2, 77.3]	10/15 (10)	80.1 [66.9, 90.8]
100	6/10 (5)	81.3 [68.8, 93.3]	4/10 (4)	77.5 [67.2, 88.2]	10/10 (9)	97.5 [96.7, 98.2]
200	4/8 (3)	80.6 [68.7, 92.3]	2/8 (2)	78.3 [71.0, 86.5]	8/8 (7)	97.6 [96.9, 98.2]

The cap matters (Table 3). On 21 development instances the original procedure admitted no plan, and on 17 of them the shortest-path bound proved that a customer had become unreachable at a large buffer. Capping raised the instances with a passing plan from 32 to 42 and pooled service by 7.7 [3.5, 12.5] points, and at 100 and 200 customers the capped procedure found a passing plan on every instance. The confirmatory studies repeated these findings (Table 4). Seven of eight primary hypotheses were confirmed: screening beat 20% padding by 15.7 points on new instances, capping added 11.7 points, and with HGS the capped procedure beat padding by 28.0 points. The one hypothesis not confirmed (G2) was that falling back to 20% padding instead of the slack-aware plan would help the capped OR-Tools procedure; it rarely needed a fallback. The best-screened fallback, tested in study 3B, raised pooled service by 7.3 and 4.2 points for the two solvers.

6.2 Fixed padding at other levels

A 20% pad may simply be the wrong amount, so Table 5 and Figure 2 repeat the comparison with padding from 0 to 50%. Over all instances no level came close to the capped procedure: the best, 30%, pooled 66.7% against 86.6%. More padding did not keep helping, because heavy padding makes many instances impossible to plan; at 50% the padded planner found a plan on only 26 of 53 instances. These failures are mostly real rather than failures of the search: of the 65 missing padded plans across all levels, 43 are

Table 4. The eight primary hypotheses of the three confirmatory studies. Each was fixed and publicly timestamped before its instances were solved. Estimates are differences in validated service probability (percentage points) with 97.5% paired bootstrap intervals over instances; a hypothesis is confirmed when its interval lies above zero.

Study		Contrast	Estimate [interval]	Decision
1	H1	Original procedure minus 20% padding (50 instances)	15.7 [7.5, 23.6]	confirmed
1	H2	Capped minus original procedure (53 instances)	11.7 [5.9, 17.9]	confirmed
2	G1	Capped procedure with HGS minus 20% padding with HGS (49 instances)	28.0 [23.0, 32.8]	confirmed
2	G2	Conservative minus slack-aware fallback, capped OR-Tools procedure	0.4 [−0.7, 2.1]	not confirmed
3A	R1	Screened robust model minus capped OR-Tools procedure (90 instances)	6.7 [3.0, 11.1]	confirmed
3A	R2	Screened robust model minus capped HGS procedure (45 instances)	6.7 [2.2, 12.2]	confirmed
3B	F1	Best-screened minus default fallback, capped OR-Tools procedure	7.3 [2.3, 13.2]	confirmed
3B	F2	Best-screened minus default fallback, capped HGS procedure	4.2 [1.2, 8.0]	confirmed

Table 5. Padding every trip by a fixed percentage on the 53 development instances (exploratory). *Plans*: instances on which the padded planner found a plan. *All*: pooled service over all instances, an instance without a plan scoring zero. The next three columns use only the instances with a padded plan: the padded plans' service, the capped screening procedure's service on the same instances, and the difference (padding minus screening, percentage points, 95% paired bootstrap interval). Distance (%) and vehicles are also padding minus screening on those instances. Over all instances the capped procedure pooled 86.6%.

		All	Instances with a padded plan				
Padding	Plans	Pooled	Padded	Screened	Difference	Distance	Vehicles
0%	53/53	21.9	21.9	86.6	−64.7 [−70.2, −58.8]	−7.1	−0.89
10%	51/53	43.7	45.4	88.4	−42.9 [−48.0, −37.4]	−6.2	−0.59
20%	46/53	56.8	65.4	89.4	−23.9 [−29.2, −18.2]	−4.8	−0.22
30%	43/53	66.7	82.2	90.3	−8.1 [−13.1, −2.4]	−3.1	0.21
40%	34/53	58.8	91.6	88.5	3.1 [−2.8, 10.0]	−1.0	0.47
50%	26/53	47.1	96.1	87.9	8.2 [1.2, 16.1]	0.9	0.77

provably infeasible, because at the padded speed some customer cannot be reached within its window or within the shift (at 50%, 18 of 27); the other 22 are unresolved. Where it did find one, the plan was reliable: on those instances 50% padding served 96.1% against 87.9% for screening, and used 0.77 more vehicles. A planner who fixes one padding level must choose between unreliable plans at low levels and missing plans at high ones.

6.3 Certification

Table 6 applies the certificate to every configuration in every study. The capped procedure was certified on 40 of 53 development instances and on 45 and 45 of 53 in the two confirmatory studies; its HGS version on 46 and 42. Twenty percent padding was never certified on a synthetic instance (0 of 238 plans): its plans are often good, but rarely good enough to show 95% with confidence. On the Amazon routes the picture depends on the delay model, which we return to below. Across the table, 666 plans were certified. On 5000 fresh days, none of them failed its certificate, and only 5 had a fresh success rate below 0.95,

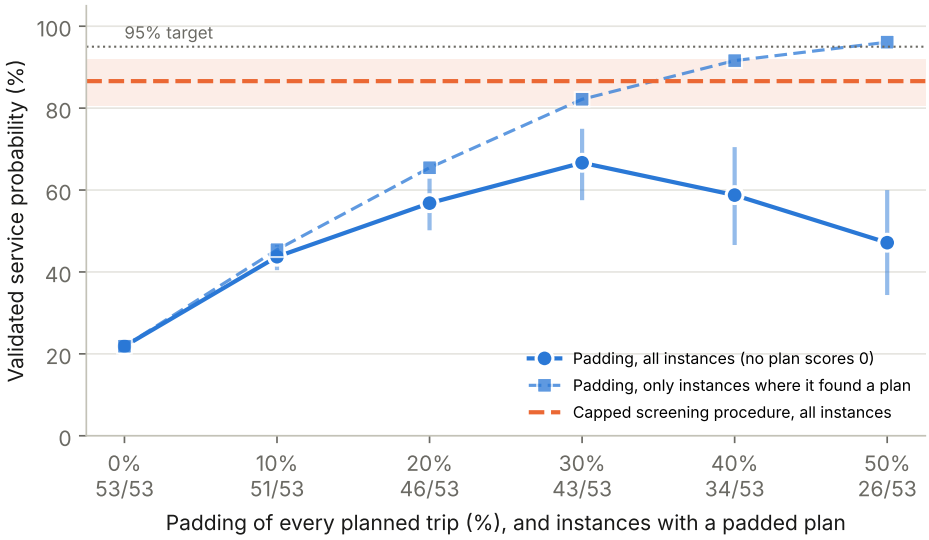


Figure 2. Validated reliability against fixed padding on the 53 development instances. Solid line: all instances, a missing plan scoring zero; dashed line: only instances where a padded plan exists (counts under the axis). Orange line and band: the capped procedure over all instances, with its 95% bootstrap interval.

Table 6. Certified plans: plans whose exact one-sided 95% lower bound on the success probability, from the 5000 validation days, is at least 0.95, out of the plans the configuration produced. Dev., Conf. 1 and Conf. 2: development and first two confirmatory studies (53 instances each); Amazon: 40 routes under the assumed delay model and under the two models fitted to New York traffic. On a further 5000 fresh days, none of the 666 certified plans in the table failed its certificate (exact one-sided 95% upper bound below 0.95).

Configuration	Dev.	Conf. 1	Conf. 2	Amazon	NYC d.	NYC all
20% padding (OR-Tools)	0/46	0/50	0/45	0/28	28/28	4/28
Original procedure	30/53	31/53	28/53	22/34	33/34	29/34
Capped procedure	40/53	45/53	45/53	22/34	33/34	29/34
Screened conservative	25/53	28/53	26/53	24/34	30/34	23/34
20% padding (HGS)	0/48	–	0/49	–	–	–
Capped procedure (HGS)	46/53	–	42/53	–	–	–

all within sampling error of it. The theorem would have allowed about 5% failures; the observed rate is lower because most certified plans are well above the target. Because the fresh days come from the same delay model, this test checks the implementation and the statistics, not the model; the test that matters in practice is the one in Section 6.7.

6.4 The cost of reliability

Table 7 and Figure 3 ask which reliability each family can reach on an instance, and at what cost. At 90% the buffer family contains a qualifying plan on 50 of 53 instances and the padding family on 33; at 95% on 46 and 23; with the exact lower bound at 95% on 45 and 20; and at 99% on 29 and 0. Where both families reach 95%, the cheapest buffered plan used 0.64 fewer vehicles [0.23, 1.09] but 1.5% more distance [0.2, 2.9] than the cheapest padded plan. At reliability levels of 80% or less, buffers were cheaper in both. Padding buys reliability by slowing every trip, which eventually costs vehicles; a buffer spends time only where the internal deadline binds.

Table 7. The cost of reliability on the 53 development instances. For each level, the number of instances on which the buffer family (13 capped candidates) and the padding family (0 to 50%) contain a plan whose validated success probability reaches it, and, where both do, the paired difference in cost between the cheapest such plans (buffers minus padding): distance in percent of the plain distance plan, and vehicles. 95% bootstrap intervals over instances. *Exact bound*: the Clopper–Pearson lower bound, rather than the estimate, must reach 95%; because the plan is chosen on the same days, this row is descriptive and is not a certificate in the sense of Theorem 1.

	Instances reached			Buffers minus padding	
Reliability	Buffers	Padding	Both	Distance (%)	Vehicles
50%	52	47	47	−1.0 [−1.7, −0.3]	−0.45 [−0.72, −0.19]
70%	51	43	43	−1.4 [−2.3, −0.5]	−0.65 [−1.05, −0.30]
80%	51	42	42	−1.2 [−2.1, −0.4]	−0.79 [−1.10, −0.50]
90%	50	33	32	0.1 [−1.2, 1.5]	−0.75 [−1.22, −0.34]
95%	46	23	22	1.5 [0.2, 2.9]	−0.64 [−1.09, −0.23]
95%, exact bound	45	20	19	1.8 [0.3, 3.4]	−0.58 [−1.00, −0.21]
99%	29	0	0	–	–

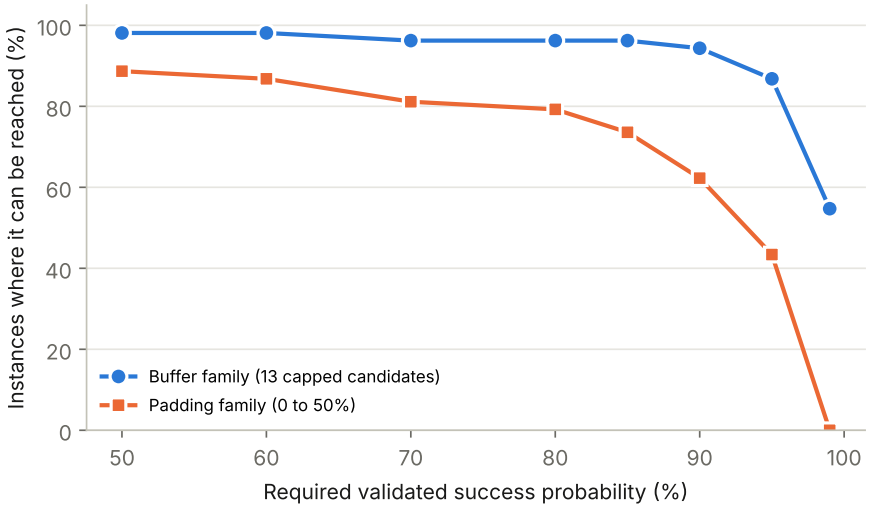


Figure 3. Share of the 53 development instances on which some plan of each family reaches a required validated reliability.

6.5 Benchmark instances

On R1_2_1 and R1_2_2, where all or most windows are only 10 time units wide, neither the anchor solves nor HGS at padded speed found a plan. Screening admitted a plan on 17 of the other 18 instances (Table 8), and on those instances the capped HGS procedure exceeded 20% padding by 15.1 [11.0, 19.1] points; over all 20 it pooled 87.2% against 73.6%.

6.6 Real delivery routes

Table 9 gives the delay model fitted to New York traffic. It differs from the assumed one in one important way: the rush hours were not slower than the middle of the day. Trips between 07:00 and 09:00 were 16% faster and trips between 17:00 and 19:00 6% faster than between 09:00 and 17:00. The assumed spread of 0.20 lies between the two fitted values.

On the Amazon routes (Table 10), the capped procedure was more reliable than 20% padding under every delay model: by 13.6 [3.9, 24.4] points under the assumed model, 13.9 [3.8, 26.2] with the fitted

Table 8. Gehring–Homberger instances with 200 customers (exploratory, HGS pipeline): instances, instances where the four single-criterion solves that set the objective's scaling ranges (the anchors) all found a plan, instances admitted by screening, and pooled validated service-event probability (%) of the HGS nominal plan, HGS conservative speed and the HGS capped procedure with conservative fallback. A missing plan scores zero.

Class	Inst.	Anchors	Admitted	Nominal	Conservative	Capped
R1	10	8	7	42.1	68.0	77.0
RC1	10	10	10	27.6	79.3	97.4

Table 9. The delay model fitted to 6.2 million weekday New York yellow-taxi trips (January–March 2024). Time-of-day multipliers of travel time relative to 09:00–17:00, from zone pairs with at least 20 trips in both periods; the spread σ and the day-common share ρ of the random factor are estimated in two ways (see text).

Period	Assumed	Fitted	Zone pairs
00:00–07:00	0.9	0.62	2,669
07:00–09:00	1.2	0.84	2,424
09:00–17:00	1.0	1.00	4,603
17:00–19:00	1.3	0.94	3,237
19:00–24:00	0.9	0.76	3,865
σ	0.20	0.11 (day to day), 0.27 (all)	
ρ	0.50	0.40 (day to day), 0.06 (all)	

day-to-day variation and 20.6 [10.9, 31.4] with all fitted variation. Much of this gap comes from planning: padding produced no plan on 12 routes, against 6 for the other configurations. On the 28 routes where both had a plan, the two were level under the assumed model (1.6 [−4.4, 7.2] points), screening was ahead with all fitted variation (8.8 [4.6, 13.3]), and with the milder day-to-day variation padding was slightly more reliable (−1.2 [−1.9, −0.6]); under that model 28 of the 28 padded plans were certified (Table 6). When no candidate passed, the capped procedure's slack-aware fallback was weaker than padding (56.8% against 62.9% under the assumed model), which screened conservative avoids because its fallback is the padded plan.

On 6 routes with 22 to 36 customers no configuration found a plan, although the driver's own order is feasible by construction, so these are limits of the solver's search. A longer search changes part of the comparison. Given 60 seconds per attempt instead of 5, OR-Tools still found no nominal plan on these 6 routes, but it found a 20% padded plan on 4 of the 12 routes where padding had none. Re-scoring padding with these plans, while keeping the screened procedures at their 5-second budget, the capped procedure's lead became 5.6 [−3.1, 14.5] points under the assumed model, 4.0 [−5.7, 14.2] with the fitted day-to-day variation and 12.1 [2.9, 21.8] with all fitted variation. On the real routes, therefore, screening is clearly more reliable than padding only under the noisier fitted model; under the other two the advantage is not statistically distinguishable once padding is given more search time. What the screen adds on every model is the certificate (Table 6).

6.7 When the delay model is wrong

A firm screens with the delay model it has, and real delays differ. The top of Table 11 takes the Amazon plans chosen under the assumed model and judges them, on fresh days, under the models fitted to New York traffic. They stayed ahead of padding at the same 5-second budget, by 12.3 [1.6, 24.9] and 16.9 [6.5, 28.6] points for the capped procedure, a margin that a longer search for padding would reduce

Table 10. Real delivery routes: 40 Amazon routes from five US cities (exploratory), under the assumed delay model and under the two delay models fitted to New York taxi trips (Table 9). *Plans*: routes on which the configuration produced a plan (the same under every delay model). Pooled validated service probability (%) with a 95% bootstrap interval over routes; a route without a plan scores zero. *Admitted*: routes on which a candidate passed the screen.

		Pooled service under the delay model		
Configuration	Plans	Assumed	NYC, day to day	NYC, all variation
Nominal plan (no protection)	34/40	41.4 [35.1, 47.1]	64.9 [55.4, 73.5]	46.9 [39.6, 53.8]
20% padding	28/40	59.0 [46.6, 70.8]	69.5 [54.7, 82.2]	59.5 [46.8, 71.8]
Capped screening procedure	34/40	72.6 [61.2, 82.9]	83.4 [71.3, 93.3]	80.1 [68.5, 90.2]
Screened conservative	34/40	75.3 [63.6, 85.7]	83.2 [71.3, 93.3]	77.3 [65.9, 87.5]
Admitted, capped procedure		24	34	30
Admitted, screened conservative		25	28	26

Table 11. When the delay model is wrong. Top: the 40 Amazon routes, judged on 5000 fresh days under each delay model fitted to New York traffic. *Wrong*: plans chosen when the screen used the assumed model; *Right*: plans chosen when it used the fitted model. Pooled service (%), and certificates (granted under the model used for screening) that still hold. Bottom: plans certified under the assumed model on the synthetic instances of the development and confirmatory studies, judged under delay models that differ from it; certificates that still hold and the mean service (%) of the certified plans.

	NYC, day-to-day variation			NYC, all variation		
	Wrong	Kept	Right	Wrong	Kept	Right
20% padding	69.5	–	69.5	59.5	–	59.5
Capped procedure	81.8	22/22	83.4	76.4	22/22	80.1
Screened conservative	83.4	24/24	83.1	77.3	24/24	77.4

	Capped (OR-Tools)		Capped (HGS)	
True delay model	Kept	Service	Kept	Service
Assumed model (new days)	130/130	97.6	88/88	97.5
Heavy-tailed delays	130/130	97.5	88/88	97.3
More noise in the peaks	102/130	95.9	66/88	95.7
Random incidents	44/130	93.3	17/88	92.6
Stronger morning peak	52/130	93.7	22/88	93.2

(Section 6.6). Screening with the right model instead added only 1.6 [−0.6, 4.4] and 3.7 [0.3, 7.5] points. Every certificate granted under the assumed model still held under both fitted models (22 of 22 for the capped procedure and 24 of 24 for screened conservative).

This is the favourable direction, because the assumed model is harsher than New York traffic in the morning. The bottom of the table shows the other direction, on the synthetic instances, with delay models chosen before the analysis. Certificates survived heavy-tailed delays (130 of 130) and mostly survived more noise in the peaks (102 of 130). They often failed when 3% of trips suffered an incident that doubled their time (44 of 130 kept) and when the morning peak was stronger than planned (52 of 130 kept), although the certified plans still averaged 93.3% and 93.7% reliability under those models, against 49.6% and 36.8% for 20% padding over all 159 synthetic instances. The certificate is therefore exactly as good as the delay model's pessimism: it carries over to milder or differently shaped delays, but not to delays that are systematically worse than modelled.

6.8 Comparison with robust optimization

A budgeted robust model plans against worst-case travel times on a limited number of trips. Without a reachability correction it failed on 16 of 45 development instances at its most protective setting, for the same reason as the uncapped procedure. With an analogous correction and the same screen, the robust plans were more reliable than both screened procedures in study 3 part A (by 6.7 and 6.7 points; Table 4), but used 0.42 more vehicles per instance than the HGS procedure, and when limited to no more vehicles than the comparison plan their advantage was no longer detectable. The layer is therefore not the most reliable planner tested; its value is that it works with the solver a firm already has, and that it says how reliable the plan is.

7 Implications for planning systems

For a firm, the layer sits between the routing solver and dispatch. It needs no change to the solver, only the ability to tighten internal deadlines and to simulate plans, and screening a candidate took 3 to 24 ms against several seconds to generate it. Four practical rules follow from the results. First, report the certificate, not only the plan: a certified plan comes with a guarantee that padding cannot give, and an uncertified one is a warning that the day is at risk. Second, use the capped deadlines; without them large buffers can make the problem infeasible for reasons that have nothing to do with the fleet. Third, choose the fallback deliberately. The best-screened candidate was the confirmed improvement in the synthetic studies, and on the Amazon routes falling back to the padded plan was safer than the unbuffered one; in either case the plan returned is not certified and should be flagged. Fourth, certify against a delay model that is at least as harsh as reality. The certificate held when real delays were milder than modelled and failed when they were harsher, so the delay model should err on the pessimistic side, for example by including incidents and a strong peak, and should be re-estimated from the fleet's own trip records.

8 Limitations

The main studies use synthetic instances and an assumed delay model; the benchmark and Amazon experiments show that the findings are not specific to the generator or to the assumed model, but the Amazon windows are assumed, and the fitted delays come from New York taxis rather than from the cities where the routes were driven. The certificate is relative to the simulated model, as Section 6.7 shows. All analyses beyond the eight primary hypotheses are exploratory. The advantage depends on the planner's objective: with equal weights on the four criteria it fell to 6.3 points, with an interval that included zero. Vehicles always leave at their planned times, and breakdowns and same-day re-planning are not modelled. Finally, OR-Tools with a 5-second limit missed feasible plans on some Amazon routes; with a longer search for padding, the reliability advantage on the real routes was clear only under one of the three delay models, so the real-route evidence supports the certificate more strongly than it supports a reliability gain.

9 Conclusion

A firm can add a reliability guarantee to the routing solver it already uses. Asking the solver for a few plans with capped internal buffers, screening them by simulation and certifying the chosen plan on independent

days gave more reliable service than fixed padding at any level on the synthetic instances, reached high reliability on twice as many instances with fewer vehicles, and produced certificates that held on fresh days without exception in our studies. The guarantee is about the delay model, so the model should be estimated from the firm's own trips and kept on the pessimistic side. Testing the layer on a fleet's own travel-time records, with its real delivery windows, is the natural next step.

Acknowledgements

The author thanks the researchers who answered methodological questions by e-mail during the project.

Use of generative AI

Anthropic's Claude and OpenAI's ChatGPT were used to write and debug the simulation and analysis code, to review that code, to run and check the statistical analyses, to search the literature, and to draft and revise the text of this manuscript, including its wording. The research question, the study design and the confirmatory protocols were the author's. Every experiment was run from the archived code, every reported number is regenerated from the archived records, and the author takes full responsibility for the content.

Disclosure statement

The author reports there are no competing interests to declare.

Funding

This research received no external funding.

Data availability statement

All code, instances, records and analysis scripts are openly available (Ali, 2026a). The Amazon route sample is redistributed under its CC BY-NC 4.0 licence; the New York taxi records are public and are not copied.

References

Hassana Abdullahi, Lorena Reyes-Rubiano, Djamila Ouelhadj, Javier Faulin, and Angel A. Juan. A reliability-extended simheuristic for the sustainable vehicle routing problem with stochastic travel times and demands. *J Heuristics*, 31(2), 2025. doi: 10.1007/s10732-025-09555-4.

A. Agra, M. Christiansen, R. Figueiredo, L. M. Hvattum, M. Poss, and C. Requejo. The robust vehicle routing problem with time windows. *Comput Oper Res*, 40(3):856–866, 2013. doi: 10.1016/j.cor.2012.10.002.

S. Ahmed and A. Shapiro. Solving chance-constrained stochastic programs via sampling and integer programming. In Z.-L. Chen and S. Raghavan, editors, *State-of-the-Art Decision-Making Tools in the Information-Intensive Age*, TutORials in Operations Research, pages 261–269. INFORMS, Hanover, MD, 2008. doi: 10.1287/educ.1080.0048.

S. Ali. Reliability-screened routing: research code and evidence archive. Zenodo, 2026a. Version v3.0, software and dataset.

S. Ali. Reliability-screened routing: research code and evidence archive. Zenodo, 2026b. Confirmatory protocol 1 freeze, commit 41fae67d9520bd5dd01fcac87e680a5a69325d10.

S. Ali. Reliability-screened routing: research code and evidence archive. Zenodo, 2026c. Confirmatory protocol 2 freeze, commit 3ba6b71288a2a51872ab6a877fa86eb28ff7671d.

S. Ali. Reliability-screened routing: research code and evidence archive. Zenodo, 2026d. Confirmatory protocol 3 freeze, commit 21ffaa8eed4e899778e14014b73533fd4d14d865.

D. Bertsimas and M. Sim. The price of robustness. *Oper Res*, 52(1):35–53, 2004. doi: 10.1287/opre.1030.0065.

F. Bomboi, C. Buchheim, and J. Pruente. On the stochastic vehicle routing problem with time windows, correlated travel times, and time dependency. *4OR*, 20(2):217–239, 2022. doi: 10.1007/s10288-021-00476-z.

L. D. Brown, T. T. Cai, and A. DasGupta. Interval estimation for a binomial proportion. *Stat Sci*, 16(2):101–133, 2001. doi: 10.1214/ss/1009213286.

Marco C. Campi and Simone Garatti. The exact feasibility of randomized solutions of uncertain convex programs. *SIAM J Optim*, 19(3):1211–1230, 2008. doi: 10.1137/07069821X.

C. J. Clopper and E. S. Pearson. The use of confidence or fiducial limits illustrated in the case of the binomial. *Biometrika*, 26(4):404–413, 1934. doi: 10.1093/biomet/26.4.404.

J. F. Ehmke, A. M. Campbell, and T. L. Urban. Ensuring service levels in routing problems with time windows and stochastic travel times. *Eur J Oper Res*, 240(2):539–550, 2015. doi: 10.1016/j.ejor.2014.06.045.

F. Errico, G. Desaulniers, M. Gendreau, W. Rei, and L.-M. Rousseau. The vehicle routing problem with hard time windows and stochastic service times. *EURO J Transp Logist*, 7(3):223–251, 2018. doi: 10.1007/s13676-016-0101-4.

H. Gehring and J. Homberger. A parallel hybrid evolutionary metaheuristic for the vehicle routing problem with time windows. In K. Miettinen, M. Mäkelä, and J. Toivanen, editors, *Proceedings of EUROGEN99*, pages 57–64, Jyväskylä, 1999. University of Jyväskylä.

M. Gendreau, G. Laporte, and R. Séguin. Stochastic vehicle routing. *Eur J Oper Res*, 88(1):3–12, 1996. doi: 10.1016/0377-2217(95)00050-X.

M. Gendreau, O. Jabali, and W. Rei. 50th anniversary invited article—Future research directions in stochastic vehicle routing. *Transp Sci*, 50(4):1163–1173, 2016. doi: 10.1287/trsc.2016.0709.

G. Ghiani and E. Guerriero. A note on the Ichoua, Gendreau, and Potvin (2003) travel time model. *Transp Sci*, 48(3):458–462, 2014. doi: 10.1287/trsc.2013.0491.

S. Davod Hosseini, Borzou Rostami, and Mojtaba Araghi. Service time window design in last-mile delivery, 2025. URL `https://arxiv.org/abs/2508.01032`. arXiv:2508.01032.

S. Ichoua, M. Gendreau, and J.-Y. Potvin. Vehicle dispatching with time-dependent travel times. *Eur J Oper Res*, 144(2):379–396, 2003. doi: 10.1016/S0377-2217(02)00147-9.

O. Jabali, R. Leus, T. Van Woensel, and T. de Kok. Self-imposed time windows in vehicle routing problems. *OR Spectrum*, 37(2):331–352, 2015. doi: 10.1007/s00291-013-0348-1.

P. Jaillet, J. Qi, and M. Sim. Routing optimization under uncertainty. *Oper Res*, 64(1):186–200, 2016. doi: 10.1287/opre.2015.1462.

Angel A. Juan, Javier Faulin, Scott E. Grasman, Markus Rabe, and Gonçalo Figueira. A review of simheuristics: extending metaheuristics to deal with stochastic combinatorial optimization problems. *Oper Res Perspect*, 2:62–72, 2015. doi: 10.1016/j.orp.2015.03.001.

A. S. Kenyon and D. P. Morton. Stochastic vehicle routing with random travel times. *Transp Sci*, 37(1): 69–82, 2003. doi: 10.1287/trsc.37.1.69.12820.

G. Laporte, F. Louveaux, and H. Mercure. The vehicle routing problem with stochastic travel times. *Transp Sci*, 26(3):161–170, 1992. doi: 10.1287/trsc.26.3.161.

X. Li, P. Tian, and S. C. H. Leung. Vehicle routing problems with time windows and stochastic travel and service times: Models and algorithm. *Int J Prod Econ*, 125(1):137–145, 2010. doi: 10.1016/j.ijpe.2010.01.013.

James Luedtke and Shabbir Ahmed. A sample approximation approach for optimization with probabilistic constraints. *SIAM J Optim*, 19(2):674–699, 2008. doi: 10.1137/070702928.

Daniel Merchán, Jatin Arora, Julian Pachon, Karthik Konduri, Matthias Winkenbach, Steven Parks, and Joseph Noszek. 2021 Amazon last mile routing research challenge: data set. *Transp Sci*, 2022. doi: 10.1287/trsc.2022.1173.

New York City Taxi and Limousine Commission. TLC trip record data: yellow taxi trip records, January–March 2024, 2024. URL `https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page`.

L. Perron and V. Furnon. OR-Tools, 2026. URL `https://developers.google.com/optimization`. Version 9.15.

T. L. Saaty. *The Analytic Hierarchy Process*. McGraw-Hill, New York, 1980.

B. Serrano, A. M. Florio, S. Minner, M. Schiffer, and T. Vidal. Contextual stochastic vehicle routing with time windows. *INFORMS J Comput*, 2026. doi: 10.1287/ijoc.2025.1189. Articles in advance.

M. M. Solomon. Algorithms for the vehicle routing and scheduling problems with time window constraints. *Oper Res*, 35(2):254–265, 1987. doi: 10.1287/opre.35.2.254.

D. Taş, N. Dellaert, T. van Woensel, and A. G. de Kok. The time-dependent vehicle routing problem with soft time windows and stochastic travel times. *Transp Res Part C Emerg Technol*, 48:66–83, 2014. doi: 10.1016/j.trc.2014.08.007.

P. Toth and D. Vigo, editors. *Vehicle Routing: Problems, Methods, and Applications*. MOS-SIAM Series on Optimization. SIAM, Philadelphia, 2nd edition, 2014. doi: 10.1137/1.9781611973594.

A. D. Vareias, P. P. Repoussis, and C. D. Tarantilis. Assessing customer service reliability in route planning with self-imposed time windows and stochastic travel times. *Transp Sci*, 53(1):256–281, 2019. doi: 10.1287/trsc.2017.0748.

T. Vidal. Hybrid genetic search for the CVRP: Open-source implementation and SWAP* neighborhood. *Comput Oper Res*, 140:105643, 2022. doi: 10.1016/j.cor.2021.105643.

E. B. Wilson. Probable inference, the law of succession, and statistical inference. *J Am Stat Assoc*, 22 (158):209–212, 1927. doi: 10.1080/01621459.1927.10502953.

N. A. Wouda, L. Lan, and W. Kool. PyVRP: A high-performance VRP solver package. *INFORMS J Comput*, 36(4):943–955, 2024. doi: 10.1287/ijoc.2023.0055.

Y. Zhang, R. Baldacci, M. Sim, and J. Tang. Routing optimization with time windows under uncertainty. *Math Program*, 175(1–2):263–305, 2019. doi: 10.1007/s10107-018-1243-y.

Yu Zhang, Zhenzhen Zhang, Andrew Lim, and Melvyn Sim. Robust data-driven vehicle routing with time windows. *Oper Res*, 69(2):469–485, 2021. doi: 10.1287/opre.2020.2043.